

AI 工具使用声明

本参赛队在竞赛过程中使用了 ChatGPT，主要用于代码检查与调试、基于 MATLAB 的科研绘图与结果可视化，以及时间标签核对、跨日补算实现、结果模板回填和支撑材料整理；参赛队已人工讨论并修改建模方案，并核验代码。详见支撑文件。

附录

A 支撑文件列表

1. AI使用说明.docx：AI工具的使用范围与方式。
2. 源码/代码/：四问模型、实验与绘图程序，以及结果导出和跨日补算程序。主版本与运行顺序见使用说明.md。
3. 结果/：result1.xlsx、result2.xlsx、result3.xlsx、result4-2.xlsx、result4-3.xlsx；问题一至问题四子目录保留原模型的汇总和明细。
4. 结果/跨日补算/：跨日预测记录、模板回填核验记录和时间口径说明。
5. 使用说明.md：目录说明、复现顺序及结果时间口径。

B 核心源码

以下列出主要计算函数。代码依赖、参数和调用入口见对应完整源码；原模型与新增跨日处理分开保存。

问题一

源码位置：源码/代码/q1_solution.py

```
def solve(p,L,V,relax=False):
    dim=6*N+1
    G,C,D,W,E,Z=0,N,2*N,3*N,4*N,5*N+1
    cost=np.zeros(dim); cost[G:G+N]=p
    lo=np.zeros(dim); hi=np.full(dim,np.inf)
    hi[C:C+N]=CAP; hi[D:D+N]=CAP
    hi[W:W+N]=V*DT
    lo[E:E+N+1]=1200; hi[E:E+N+1]=10800
    hi[Z:Z+N]=1
    a=lil_matrix((4*N+1,dim)); lower=np.full(4*N+1,-np.inf); upper=np.zeros(4*N+1)
    a[4*N,[E,E+N]]=[-1,-1]
    lower[4*N]=upper[4*N]=0
    for t in range(N):
        a[t,[G+t,C+t,D+t,W+t]]=[-1,-1,-1,-1]
        lower[t]=upper[t]=(L[t]-V[t])*DT
        a[N+t,[E+t+1,E+t,C+t,D+t]]=[-1,-1,-ETA,1/ETA]
        lower[N+t]=upper[N+t]=0
        a[2*N+t,[C+t,Z+t]]=[-1,-CAP]
        a[3*N+t,[D+t,Z+t]]=[-1,CAP]; upper[3*N+t]=CAP
    integer=np.zeros(dim); integer[Z:]=0 if relax else 1
    r=milp(cost,integrality=integer,bounds=Bounds(lo,hi),
           constraints=LinearConstraint(a.tocsc(),lower,upper),
           options={'mip_rel_gap':1e-9,'time_limit':120})
    assert r.success,(r.status,r.message)
    return r, [r.x[s:s+N].copy() for s in [G,C,D,W]],r.x[E:E+N+1].copy()
```

问题二

源码位置：源码/代码/实验/q2_fresh_20260912/fresh_engine.py

```
def plan_lp(p,net,E0,kappa):
    H=len(p);dim=5*H+1;g=0;c=H;q=2*H;w=3*H;e=4*H
    obj=np.zeros(dim);obj[:H]=p;obj[-1]=-kappa
    bounds=[(0,None)]*dim
    A=lil_matrix((2*H,dim));rhs=np.r_[net,np.zeros(H)]
    for t in range(H):
        A[t,[g+t,c+t,q+t,w+t]]=[-1,-1,-1,-1]
        A[H+t,[e+t+1,e+t,c+t,q+t]]=[-1,-1,-ETA,1/ETA]
        bounds[c+t]=bounds[q+t]=(0,CAP)
    for t in range(H+1):bounds[e+t]=(EMIN,EMAX)
    bounds[e]=(E0,E0)
    res=linprog(obj,A_eq=A.tocsr(),b_eq=rhs,bounds=bounds,method='highs')
    assert res.success,res.message
    assert abs(A@res.x-rhs).max()<TOL
    return np.maximum(res.x[:H],0)

def action(E,r,p,future,grid=GRID):
    if r<=0:
        c=max(0,min(-r,CAP,(EMAX-E)/ETA));return c,0.,0.,max(0,-r-c),E+ETA*c
    lo=max(EMIN,E-min(r,CAP)/ETA)
    knots=grid[(grid>=lo-1e-9)&(grid<=E+1e-9)]
    knots=np.unique(np.r_[lo,E,knots]);knots=np.clip(knots,lo,E)
    costs=5*p*np.maximum(r-ETA*(E-knots),0)+np.interp(knots,grid,future)
    nxt=float(knots[np.argmin(costs)])
    q=max(0,ETA*(E-nxt));e=max(0,r-q)
    return 0.,q,e,0.,nxt
```

问题三

源码位置：源码/代码/实验/q3_revision_a_20260912/q3_engine.py

```
def bellman_batch(p, plans, m, grid=GRID):
    plans=np.atleast_2d(plans);K,H=plans.shape;G=len(grid)
    V=np.zeros((K,H+1,3,G)); step=grid[1]-grid[0]
    width=int(np.floor(CAP/ETA/step+1e-10))+1
    ix=np.arange(G)[:None]-np.arange(width)[None,:]
    valid=ix>=0;ix=np.maximum(ix,0)
    for t in range(H-1,-1,-1):
        f_all=V[:,t+1] if t==H-1 else np.einsum('ij,kjg->kig',m['P'][t],V[:,t+1])
        for s in range(3):
            f=f_all[:,s];r=m['samples'][t,s][None,:]-plans[:,t,None]
            x=grid[None,None,:]
            distance=np.minimum(np.maximum(r,0),CAP)/ETA
            lo=np.maximum(EMIN,x-distance[:, :, None])
            knot=f+5*p[t]*ETA*grid
            windows=np.where(valid[None, :, :], knot[:, ix], np.inf)
            windows=np.minimum.accumulate(windows,axis=2)
            size=np.floor(distance/step+1e-10).astype(int)
            minima=windows[np.arange(K)[:None,None],np.arange(G)[None,None,:],size[:, :, None]]
            boundary=interp_batch(f,lo,grid)+5*p[t]*ETA*lo
            deficit=5*p[t]*(r[:, :, None]-ETA*x)+np.minimum(minima,boundary)
            nxt=np.minimum(EMAX,x+ETA*np.minimum(np.maximum(-r,0),CAP))[:, :, None]
            surplus=interp_batch(f,nxt,grid)
            cost=np.where(r[:, :, None]>0,deficit,surplus)
            V[:,t,s]=np.einsum('ksg,s->kg',cost,m['weights'][t,s])
    assert np.isfinite(V).all() and np.diff(V,axis=3).max()<TOL
    return V
```

问题四

源码位置：源码/代码/实验/q4_independent_codex_20260912/model.py

```
def bellman(g,mdl,grid,scalar=False):
    h,s,l=mdl.mass.shape; assert np.asarray(g).shape==(h,)
    w=np.zeros((h+1,s,len(grid)))
    for t in range(h-1,-1,-1):
        future=mdl.transition[t]@w[t+1] if t<h-1 else w[t+1]
        if scalar:
            for state in range(s):
                for j,e in enumerate(grid):
                    w[t,state,j]=sum(mdl.mass[t,state,a]*action(e,mdl.demand[t,state,a]-g[t],
                        mdl.price[t,state,a],future[state],grid)[1] for a in range(1) if
                        mdl.mass[t,state,a]>0)
            continue
        gap=mdl.demand[t]-g[t]; pp=mdl.price[t]; e=grid[None,None,:]
        lower=np.maximum(LOW,e-np.minimum(np.maximum(gap,0),RATE))[:, :, None]/ETA)
        at_lower=interpolate_rows(future,lower,grid)+5*pp[:, :, None]*(gap[:, :, None]-ETA*(e-lower))
        best=5*pp[:, :, None]*gap[:, :, None]+future[:, None, :]
        best=np.minimum(best,at_lower)
        reach=np.minimum(np.maximum(gap,0),RATE)/ETA
        for offset in range(1,min(len(grid),int(np.ceil(RATE/ETA/(grid[1]-grid[0])))+1)):
            delta=offset*(grid[1]-grid[0])
            trial=future[:, None, :-offset]+5*pp[:, :, None]*(gap[:, :, None]-ETA*delta)
            valid=reach>=delta
            best[:, :, offset:]=np.minimum(best[:, :, offset:],np.where(valid[:, :, None],trial,np.inf))
        en=np.minimum(HIGH,e+ETA*np.minimum(np.maximum(-gap,0),RATE))[:, :, None]
        surplus=interpolate_rows(future,en,grid)
        costs=np.where((gap<=0)[:, :, None],surplus,best)
        w[t]=np.sum(mdl.mass[t, :, :, None]*costs,axis=1)
    return w
```

时间对齐与跨日延拓

源码位置：源码/代码/结果导出/boundary_rules.py

```
def solve_next_cycle(solver, price, load, pv, previous_end):
    fit, actions, stored = solver(price, load, pv)
    assert abs(stored[0] - previous_end) < 1e-6
    assert abs(stored[-1] - stored[0]) < 1e-6
    return fit, actions, stored
def extend_pv_hours(forecasts):
    forecasts = np.asarray(forecasts, dtype=float)
    assert forecasts.shape == (365, 4, 24)
    carried = forecasts[-1, 3, 6:]
    estimated = forecasts[-7:, 0, 18:].mean(axis=0)
    hourly = np.r_[carried, estimated]
    assert np.isfinite(hourly).all() and (hourly >= 0).all()
    return hourly
def match_template_day(today, next_first):
    today = np.asarray(today, dtype=float)
    assert today.shape == (144,) and np.isfinite(today).all()
    return today[1:].tolist() + [next_first]
```

C 时间口径与跨日计算

输入时标按十分钟区间的结束时刻解释，例如0:10表示00:00—00:10。结果模板时间栏保持原样，首段00:10—00:20对应模型第二段，末段对应次日00:00—00:10；末段不能直接用当天第一段替代。

第一问的新增边界假设是光伏预测曲线延续一个日周期，电价和负载按题设每日重复。调用原求解器重新计算下一周期，并校验其期初储电量与前一期末相接。次日首段购电量为1406.6411 kWh，储电衔接值为8550.0000 kWh；该值依赖日周期延拓假设。

第二问与4-2沿用原预测和优化方法向后补算，2026年1月1日首段为1447.4053 kWh。第三问与4-3沿用2025年12月31日18点预报覆盖次日1—18点，对19—24点采用此前7天零点发布的同一时刻预报均值延拓，再执行原插值、历史残差校准及优化，首段为1446.5880 kWh。延拓预报不是附件中实际发布的新预报，也不是实际观测。

购电表合计覆盖模板展示的00:10—次日00:10，充放电和紧急购电仍按各自模板指定的自然日统计；原论文全年费用对应自然日范围，二者不能直接混用。每行末段接入次日零点产生的计划，不将其表述为前一日零点已知的决策。4-2和4-3年末跨日费用采用预测电价。

完整跨日计算见源码/代码/结果导出/boundary_prediction.py；模板数据准备、回填和验证分别见prepare_templates.py、refill_templates.mjs与verify_templates.py。